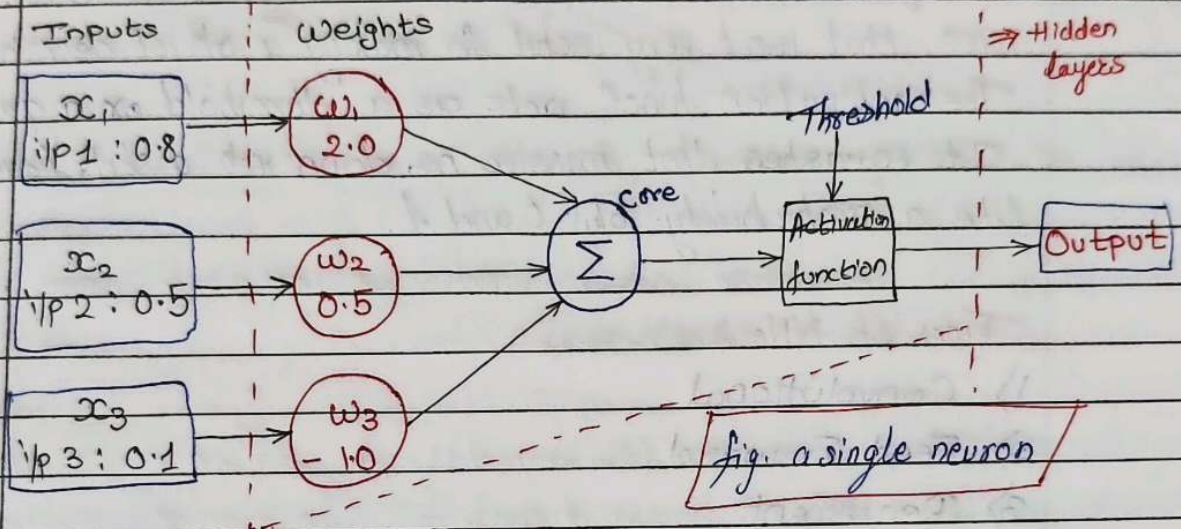


Ch-2

Intro to RNNArtificial Neuron : $\rightarrow$

- A single neuron is a basic electrical circuit or a data processing unit node.
- It takes incoming signals, applies some resistance to them, adds them up, and decides whether or not to trigger an output.



i) inputs (x): It is the raw data.

Suppose, we are trying to predict housing prices,

Input 1 might be square footage

i/p 2 might be no. of bedrooms / amenities

i/p 3 is the age of the house

ii) Weights (w): These are the amplifiers or resistors.

The n/w learns that sq. footage is highly important. so, it gets more weightage of 2.0.

It might learn that with age of house increment, price might get lower. so, a negative weightage of -1.0.

iii) Summation & bias:

The core of the neuron takes every i/p, multiplies it by its weight, and add them all together.

It then adds a bias.

iv) Activation function:

If the summation spits out a massive number, say 45000, that isn't very useful for making a strict decision.

The activation funcⁿ acts as a threshold or a gate.

It squashes that massive no. down into a std. format, like a probability b/w 0 and 1.

Types of NNs:

- 1) Convolutional
- 2) Feed Forward
- 3) Recurrent
- 4) Transformers
- 5) Generative Adversarial (GAN)

NNs?

- A single neuron can only make very basic decisions.
 - To solve complex problems, we wire hundreds or 1000s of these neurons together into layers.
 - These are also called standard feed forward NN.
- ⊗ (FNN).

How they work?

NNs work by learning through a cycle of trial & error.

If we build a n/w and give it random weights, its first prediction will be complete garbage. Its learning is simply the process of fixing those weights.

i) Forward pass: The n/w makes a guess

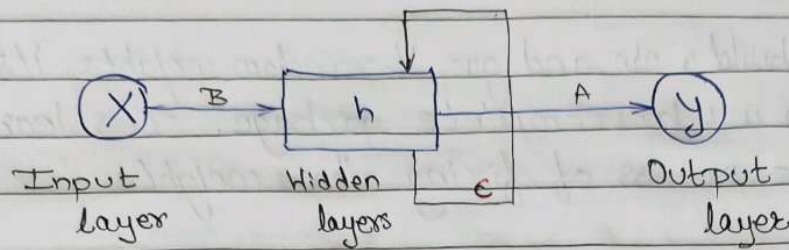
ii) Loss function: A mathematical referee that calculates exactly how wrong the guess was.

iii) Back propagation: The n/w works backward from the %p, looking at every single weight and asking, "Did you ~~concrete~~ contribute to this error? If so, turn your dial down. If you were right, turn up your dial.

It repeats the process 1000s of times, slowly walking down the mountain of error until it finds the lowest point (the perfect weights). This is called Gradient Descent.

* Recurrent Neural Networks (RNN) :->

- Type of AI designed specifically to process sequential data, e.g. sentences, audio streams, or daily stock prices, by maintaining an internal "memory" of the past.
- Unlike standard networks that treat every piece of data as an isolated event, an RNN reads info. step-by-step and continuously passes a hidden summary note forward to itself.
- This looping mechanism allows the n/w to build context and understand the present based on what happened previously.
- It often requires architectural upgrades like LSTMs (Long-short term memory) to prevent that memory from fading over very long sequences.



A, B, and C are the parameters

RNN architectural shapes →

- 1) One-to-one : Standard non-recurrent n/w
(Fixed i/p → Fixed o/p)
- 2) One-to-many : A static single i/p yields sequential o/p
e.g. Image captioning.
An image goes in, and a sequence of words comes out defining the scene.
- 3) Many-to-many (Delayed) : A sequence of i/p is completely processed before a sequence of words/o/p begins.
e.g. Machine Translation.
An RNN reads an entire English sentence to grasp the full context before it begins translating it into Hindi.
- 4) Many-to-one : A sequence of i/p yields a single static o/p.
e.g. Sentiment Analysis.
A sequence of words (a tweet) is read, and a single classification score (Positive/Negative/Neutral) is o/p.
- 5) Many-to-many (Synchronized) : Every single i/p time-step maps directly to an immediate o/p time-step.
e.g. Video Frame Classification or token-level labeling

TransformersMore on RNN:

So, RNN introduced a hidden state (h_t) - a mathematical "memory note" passed from one step to the next as it reads ~~reads~~ text word-by-word.

Mathematically,

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

↙
↓
↘

Current hidden state
Previous hidden state
current i/p
↗ bias term

$W_{hh} \rightarrow$ Hidden weight matrix $\Rightarrow$ A learned parameter that decides how much importance to give to the past memory. consider it as a volume dial for past

$W_{xh} \rightarrow$ Input weight matrix $\Rightarrow$ A learned parameter that decides how much importance to give to the brand new i/p. think of it as a volume dial for present.

* Flaw of RNN $\rightarrow$ Terrible long term memory (The Vanishing Gradient)

As NN learns by adjusting its weight backward through time, the numbers get multiplied repeatedly.

Because the n/w continuously multiplies and squashes its weight matrices and squashes it through the tanh funcⁿ at every single step, the information gets diluted.

By the time it reads 100th word, the signal from word 1 has faded to zero. It usually remembers the data from word 90 to 99.

To overcome this problem of memory loss in RNN, Researchers introduced an advanced form of RNN -

* LSTMs (Long Short-Term Memory)

— A standard RNN only has one track of memory: the hidden state (h_t).

It tries to use this single memory for both short-term and long-term context. Hence, it gets overwhelmed and forgets.

— An LSTM separates these two jobs by creating a 2nd track of memory.

- The hidden state (h_t) : $\rightarrow$ Handles the short-term memory (What is happening right now).

- The cell state (C_t) : $\rightarrow$ The new, long-term memory.

Cell state: A conveyor belt running straight down the entire length of NN.

Info flows down this belt incredibly easily, with almost no interference.

This allows long-term memory to flow/travel from the beginning of a para all the way to the end without vanishing.

The Gates Controls what goes onto the conveyor belt and what gets taken off.

They look at incoming data (new data) and strictly decide what is allowed to pass through to the long-term memory.

Date: / /

There are 3 gates, and all operates using a Sigmoid Activation Function (σ). these σ squishes nos b/w 0 and 1.

0 $\rightarrow$ "Block" $\rightarrow$ throw this away

1 $\rightarrow$ "Pass" $\rightarrow$ keep it.

i) The Forget Gate $\rightarrow$

Decide what old info on the conveyor belt is no longer needed. Erase it.

ii) The Input Gate $\rightarrow$

Decide what new info should be added to the conveyor belt. Write new important data.

iii) The Output Gate $\rightarrow$

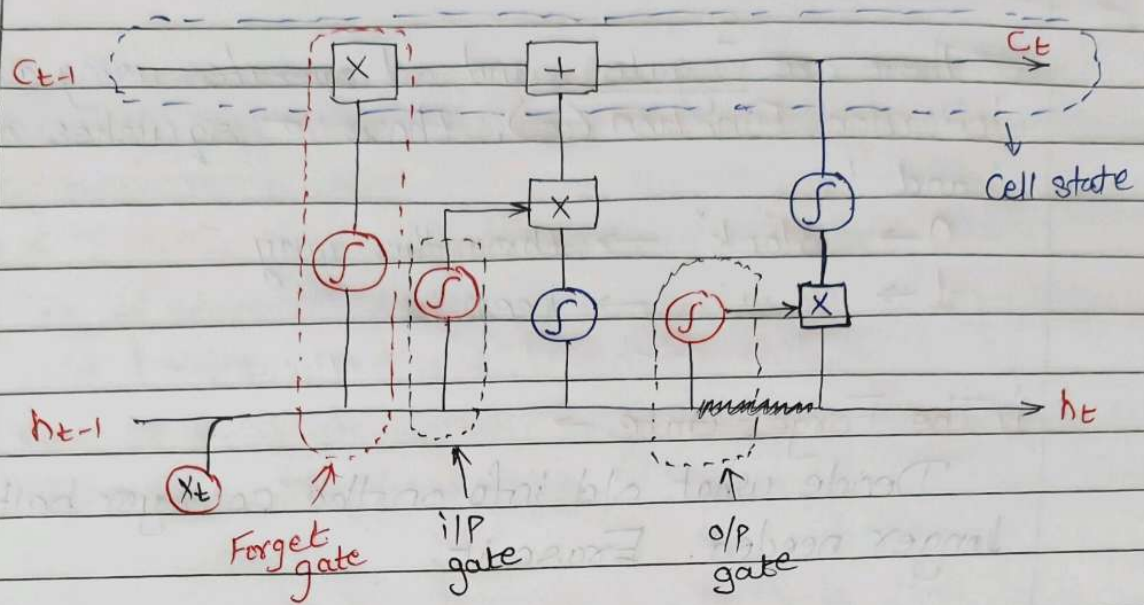
Decide what final alp of this specific step should be. i.e. Decides what memory is needed right now.

Mathematically,

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \bar{C}_t$$

The forget gate (f_t) decides what to erase from the old cell state (C_{t-1}). Then, the i/p gate (i_t) decides what new info. ($\bar{C}_t$) to add.

Date: / /



$\sigma \rightarrow$ sigmoid

$\tanh \rightarrow$ tanh

$\times \rightarrow$ Pointwise multiplication

$+$ $\rightarrow$ Pointwise addition